

주행2

재난 구조 로봇

C-1





Mapping Algorithm Review

클래스 Pseudo Code

코드 리뷰

클래스 MapWithPose(Node):

초기화 함수 `__init__()`:

- 부모 클래스(Node) 초기화
- 맵 구독자 생성 (토픽: '/map')
- 포즈 구독자 생성 (토픽: '/pose')
- 목표 지점 발행자 생성 (토픽: '/goal_pose')

- 초기 변수 설정:

```
is_init_pose = False # 초기 포즈 확인 여부
pose = None         # 현재 포즈 저장
goal_start = False  # 목표 시작 여부
map = None          # 맵 데이터 저장
count = 0           # 카운터 초기화
```

- Map을 자동으로 실행하기 위한 코드
- Topic Map과 포즈에 대한 정보 얻기 위한 node 생성
- Map노드: map_callback 함수로 가야할 위치 선정
- Pose: 로봇의 현재 위치를 탐지
- /goal_pose : 목표 지점에 대한 publisher



Mapping Algorithm Review

Mapping_callback Pseudo Code

함수 map_callback(self, msg):

만약 포즈 정보, 메시지가 없으면 종료
만약에 목표가 이미 설정되었으면 종료

맵 이미지 생성 --> data_to_image

경계점과 목표 지점 찾기:

points = 경계점 찾기 --> find_boundary

goal_point = 목표 지점 찾기--> find_goal

만약 경계점이 없으면

count 증가

만약 count > 10 이면:

맵핑 완료 및 초기 위치로 복귀

함수 종료

목표 지점이 있으면:

목표 지점 발행

그렇지 않으면:

경고 로그 출력

코드 리뷰

Map데이터를 기반으로 goal pose를 찾는 알고리즘

- 1. 포즈 정보나 맵 데이터 정보가 없으면 종료
- 2. 맵을 이미지로 변경 (data_to_image)
- 3. find_boundary(map_img) # 경계선 찾기
- 4. find_goal 경계선 기반으로 목표 지점 찾기
- 5. 경계점이 없으면 count 증가 (term을 주기 위해) 후 복귀
- 6. 없으면 경고 출력



Mapping Algorithm Review

코드 리뷰

MapToImage Pseudo Code

함수 data_to_image(data):

- 1D 맵 데이터를 2D 이미지로 변환
- 입력: data (1차원 배열)

작업:

1. 1D 배열을 (height, width) 크기의 2D 배열로 변환
2. 배열의 값이 -1인 경우, 255로 변경
3. 배열의 데이터 타입을 uint8로 변환 (이미지로 사용 가능하게)

반환: 변환된 2D 이미지

Map데이터를 이미지 데이터 변환하기 위한 함수

수

- -1인 경우 255으로 변경
- Uint8로 변환



Mapping Algorithm Review

코드 리뷰

FindBoundary Pseudo Code Pseudo Code

1. 경계 검출:
 - a. 이미지의 각 픽셀에 대해 아래 조건 확인:
 - 현재 픽셀 값이 0이고, 상/하/좌/우 중 하나가 255인 경우 경계로 판단.
 - b. 경계 정보를 저장할 새로운 이미지 생성:
 - 경계 위치에 대해 255 값을 설정, 나머지는 0으로 설정.
2. 경계 확장 (Dilation):
 - a. 커널(3x3) 생성 하여, 경계선 두께를 확장.
3. 연결된 구성 요소 분석:
 - a. 확장된 경계선 이미지에서 연결된 구성 요소(Label)를 식별.
 - b. 각 구성 요소에 고유한 Label을 할당.
4. 중심 좌표 계산:
 - a. 연결된 구성 요소의 각 Label에 대해:
 - Label에 속한 픽셀 좌표를 찾는다.
 - 노이즈 제거: 픽셀 수가 최소 임계값(min_size) 이상인 경우만 처리.
 - 중심 좌표(x, y) 계산:
 - x = 픽셀의 가로 좌표 평균값
 - y = 픽셀의 세로 좌표 평균값
 - 중심 좌표를 리스트에 추가하여 반환

픽셀별로 이미지의 경계를 탐지하는 함수

1. 경계를 0과 255 사이로 판단
2. 현재 픽셀 값이 0인 곳으로 판단 (갈 수 있는 곳)
3. 중심좌표를 구하여 충돌 방지



Mapping Algorithm Review

FindGoal and is_safe_goal Pseudo Code

코드 리뷰

Goal Points를 각각 탐색 진행
각 point에 대하여
If min_거리보다 거리가 짧고 안전한 구역이라면 :
 최종 목적지 지정
 min_거리를 distance로 지정

최종 목적지 지정

goal point 후보 중 안전 구역 요건 충족 및 최단 거리
의 point를 선정하는 함수

맵 데이터와 safety_radius를 지정하여 안전 구역 안
에 있는지 확인 (is_safe_goal)합니다.



Mapping Algorithm Review

FindGoal and is_safe_goal Pseudo Code

코드 리뷰

Is_safe_goal

Goal 지점을 맵 위의 좌표로 변환

Function is_goal_safe(goal_x, goal_y, safety_radius=0.6, obstacle_value=100):

- Iterate y from (map_y - radius_pixels) to (map_y + radius_pixels + 1):
- Iterate x from (map_x - radius_pixels) to (map_x + radius_pixels + 1):
 - If (x, y) is outside the map boundary:
 - Skip to the next pixel
 - If the pixel value at (x, y) is equal to obstacle_value:
 - Return False (목표 지점 주변에 장애물 존재)

3. 결과: 주변에 장애물이 없는 경우

- Return True

1. Goal 지점을 맵 위의 좌표로 변환하고

safety_반경을 지정

2. 반경안에 100이 있으면 False, 아니면 True

코드 리뷰

Detection And Map marking

C-1





Detection And Map marking Review

코드 리뷰

SIFTDetector Pseudo Code

Class __init__

Function __init__(ori_img, cap_img, types):

Input:

- ori_img: 템플릿 이미지 (원본)
- cap_img: 캡처된 이미지
- types: 매칭 기준 타입 (1: ext, 2: man)

Steps:

- ori_img와 cap_img를 저장
- types 값을 저장
- result 초기값을 False로 설정
- detect() 함수를 호출하여 이미지 매칭 수행

1. 이미지 탐지를 위한 클래스
2. 템플릿 이미지와 types, 캡처된 이미지를 받음
3. Detect함수를 통한 이미지 매칭



Detection And Map marking Review

SIFTDetector Pseudo Code

Function detect():

Purpose:

- SIFT 알고리즘을 통해 두 이미지 간 매칭 수행 및 객체 탐지

Steps:

1. SIFT 생성 및 특징점 추출

- ori_img와 cap_img에서 키포인트(kp)와 디스크립터(des) 추출
- 키포인트나 디스크립터가 부족하면 종료(result=False)

2. 특징 매칭 수행

- match_features() 호출하여 좋은 매칭점 계산

3. 매칭점 개수 검증

- types에 따라 매칭점 기준값 설정 (1: 35, 2: 100)
- 매칭점이 기준보다 적으면 종료(result=False)

4. Homography 계산

- compute_homography() 호출하여 변환 행렬 계산
- 예외 처리: Homography 계산 실패 시 종료(result=False)

5. 투영 오류 검증

- validate_projection_error() 호출하여 투영 오류 확인
- 투영 오류가 임계값을 초과하면 종료(result=False)

6. 중심 좌표 및 크기 계산

- calculate_center_and_size() 호출하여 중심 좌표와 크기 계산
- 계산 실패 시 종료(result=False)

7. 결과 저장

- bounds와 points 저장 후 result=True로 설정



Detection And Map marking Review

SIFTDetector Pseudo Code

Function match_features(des1, des2, threshold=0.8):

Purpose:

- FLANN 매칭을 통해 특징점 매칭 수행

Steps:

1. FLANN 매치 생성
 - 알고리즘과 검색 파라미터 설정
2. KNN 매칭 수행
 - des1과 des2를 비교하여 후보 매칭점 리스트(matches) 생성
3. Lowe's ratio test 적용
 - 후보 매칭점 중 좋은 매칭점만 추출
 - 두 매칭점 거리 비율이 threshold보다 작은 경우 선택
4. 반환
 - 좋은 매칭점 리스트(good_matches) 반환

- Match_features를 통하여 좋은 매칭점 리스트를 origin image와 real image로 변환



Detection And Map marking Review

Selector Pseudo Code

코드 리뷰

Input:

points: [(x1, y1), (x2, y2), ...] - 입력 포인트 리스트.

bounds: ((center_x, center_y), (width, height)) - 선택 경계 정보.

Execution:

1. 초기화:

- 입력된 points와 bounds를 저장.
- select() 호출로 30개 임의의 좌표 선택.

2. 호출 시:

- __call__() 메서드를 통해 현재 선택된 points를 반환.

Output:

- 선택된 30개의 포인트 리스트.

- Matching된 point들 중 30개 임의 추출하는 코드
- 30개 데이터는 PNP에 사용될 좌표들



Detection And Map marking Review

Pointer Pseudo Code

코드 리뷰

Function `__init__(camera_matrix, dist_coeffs, tf_map_camera)`:

Purpose:

- Pointer 클래스 초기화:
카메라 매트릭스, 왜곡 계수, 맵에서 카메라로의 변환 행렬 설정.

Input:

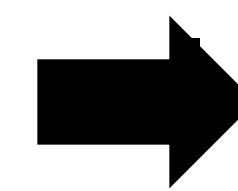
- `camera_matrix`: 카메라의 내부 파라미터 행렬 (3x3).
- `dist_coeffs`: 카메라 렌즈의 왜곡 계수.
- `tf_map_camera`: 맵 좌표계에서 카메라 좌표계로의 변환 행렬.

Steps:

1. 입력된 `camera_matrix`, `dist_coeffs`, `tf_map_camera`를 멤버 변수로 저장.

- 카메라 매트릭스, 왜곡 계수, map to camera를 인자로 받음

- 이 때, `tf_map_camera`는 tf listener로 받는 것을 1차 목표



실제로는 변환 문제가 발생하여 `tf_echo`로 받은 값 사용



Detection And Map marking Review

코드 리뷰

Pointer Pseudo Code

Function `__call__`(pixel_list, scale):

Input:

- pixel_list: [(x1, y1), (x2, y2), ...], 이미지의 픽셀 좌표 리스트.
- scale: (width, height), 이미지의 실제 크기 (단위: meter).

Steps:

1. pixel_list와 scale을 멤버 변수로 저장.
2. pixel_to_scale() 호출:
 - 픽셀 좌표를 실제 크기 기반 3D 좌표로 변환.
3. pnp_and_tf() 호출:
 - 3D 포인트와 2D 픽셀 간 관계를 기반으로 변환 행렬 계산.
4. final_tf() 호출:
 - 최종 변환 행렬 계산 (맵 → 이미지).
5. pointer() 호출:
 - 맵 좌표 반환.

Output:

- (avg_x, avg_y): 맵 상의 평균 좌표.

- 이미지의 30개 좌표를 받아 steps에 따라 진행
- 이 때, 이미지 크기는 사전에 정의되어 있는 것을 사용



Detection And Map marking Review

Pointer Pseudo Code

코드 리뷰

함수 pixel_to_scale:

```
# 변환된 3D 좌표를 저장할 빈 리스트를 초기화
scale_list를 빈 리스트로 초기화
# matches_list에 있는 (x, y) 픽셀 좌표를 하나씩 처리
matches_list의 각 match에 대해 반복:
    # match에서 x와 y 값을 추출
    # 이미지 크기 대비 실제 맵 크기를 기준으로 x 좌표를 변환
    x = x / scale_img[0] * scale_real[0]

    # 이미지 크기 대비 실제 맵 크기를 기준으로 y 좌표를 변환
    y = y / scale_img[1] * scale_real[1]
    # z 좌표는 0으로 설정 (2D 좌표를 3D로 확장)
    z = 0
    # 변환된 (x, y, z) 좌표를 scale_list에 추가
    scale_list에 [x, y, z] 추가
# 변환된 3D 좌표 리스트를 클래스 변수 scale_list에 저장
self.scale_list를 scale_list로 설정
```

- 이미지의 30개 픽셀 좌표와 그와 매칭되는 origin image 좌표 계산
- Pixel_to_scale로 3d 좌표 완성



Detection And Map marking Review

Pointer Pseudo Code

코드 리뷰

Function pnp_and_tf():

Purpose:

- 3D 점과 2D 픽셀 매칭을 기반으로 카메라와 이미지 간 변환 행렬 계산.

Steps:

1. object_points: scale_list를 numpy 배열로 변환.
2. image_points: pixel_list를 numpy 배열로 변환.
3. solvePnPRansac 호출:
 - 3D object_points와 2D image_points로 카메라 포즈 추정.
 - 반환값: 회전 벡터(rvec), 이동 벡터(tvec).
4. 회전 벡터(rvec)를 회전 행렬로 변환(cv2.Rodrigues 사용).
5. 4x4 변환 행렬(tf_camera_image) 생성:
 - 상단 왼쪽 3x3: 회전 행렬.
 - 상단 오른쪽 3x1: 이동 벡터(tvec).

- Pnp 알고리즘을 scale_list와 pixel_list를 기반으로 진행
- Pnp 알고리즘은 cv2.solvePnPRansac 사용
- 이후 tf와 points를 dot하여 x,y를 각각 계산,
- point의 평균을 균맵에서의 좌표 계산



Detection And Map marking Review

코드 리뷰

GUI.map_to_pixel Pseudo Code

Function point_pixeling(x, y)

1. 입력 유효성 확인:

- node.map_img 또는 node.undistort_image가 None인지 확인
- 조건을 만족하지 못하면 반환하지 않음.

2. 픽셀 좌표 변환:

- $x_pixel = (x - origin_x) / resolution$
 - origin_x: 맵 원점의 x 좌표 (node.origin.position.x)
 - resolution: 맵 해상도 (meter per pixel).
- $y_pixel = (y - origin_y) / resolution$
 - origin_y: 맵 원점의 y 좌표 (node.origin.position.y).

3. 정수형으로 변환:

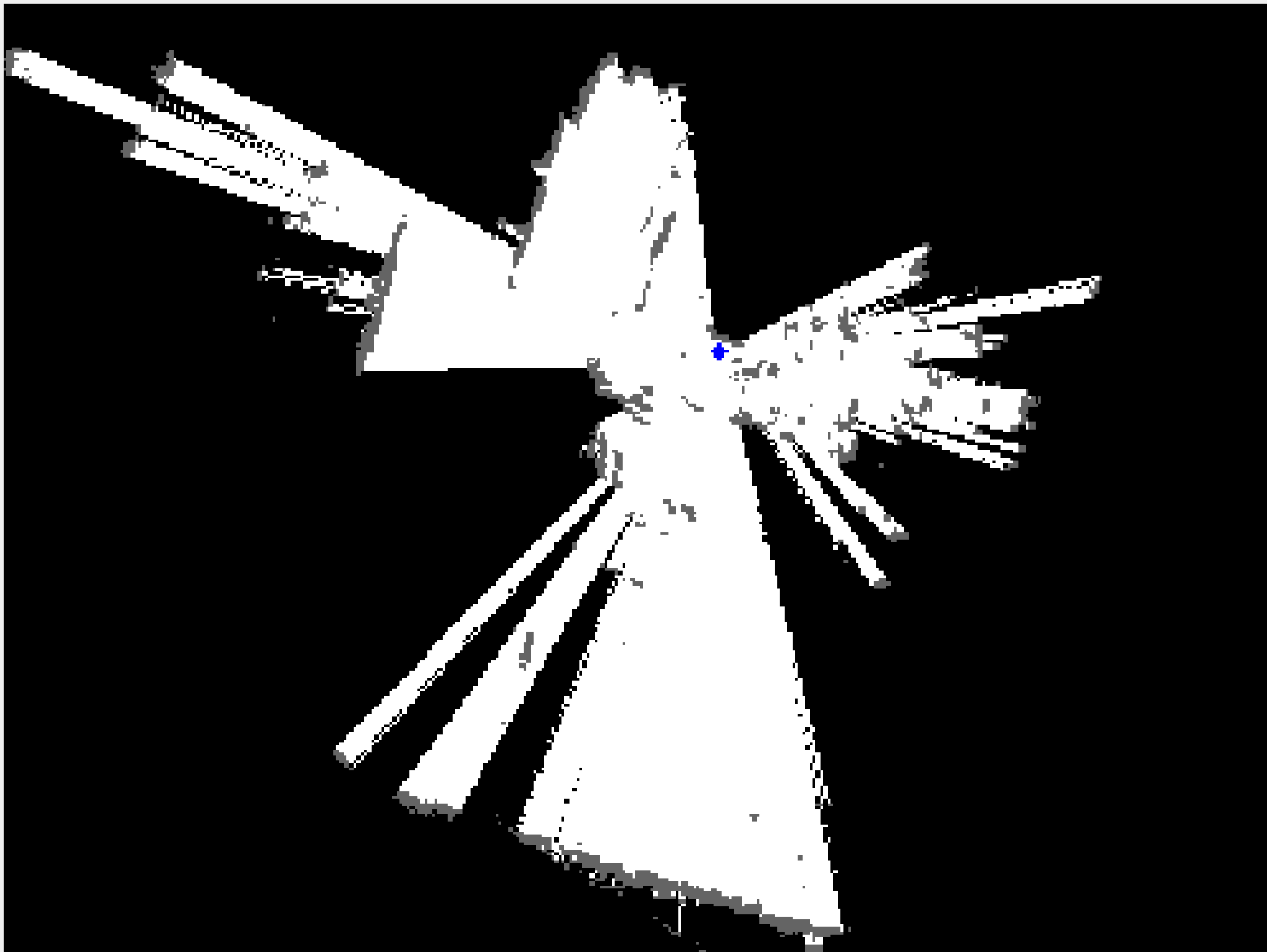
- x_pixel, y_pixel 값을 int()로 변환.

4. 결과 반환:

- (x_pixel, y_pixel)를 반환.

- Pointer에서 나온 좌표를 맵 픽셀 좌표로 변환하는 함수

▶ 시연 결과





navi2.yaml tuning value 전략

local_costmap 관련 설정

resolution (해상도) 기본값: 0.06m → 수정값: 0.01m

변경 이유: 더 세밀한 장애물 인식과 정밀한 로봇 제어를 위해 해상도를 증가시킴. 높은 해상도는 장애물 근처를 안전하게 지나갈 수 있도록 도와주지만, 계산 비용이 커짐.

inflation_radius (인플레이션 반경) 기본값: 0.45m → 수정값: 0.6m

변경 이유: 로봇의 안전 거리를 확보하기 위해 반경을 조정. 너무 큰 값은 경로 효율을 방해하고, 너무 작은 값은 충돌 위험을 증가시킴.

obstacle_max_range (장애물 최대 탐지 거리) 기본값: 2.5m → 수정값: 3.0m

변경 이유: 장애물 탐지 범위를 확장하여 더 먼 거리에서 장애물을 감지하고 회피할 수 있도록 하여 안전성을 증가시킴.



navi2.yaml tuning value 전략

global_costmap 관련 설정

obstacle_max_range (장애물 최대 탐지 거리) 기본값: 2.5m → 수정값: 3.0m
변경 이유: 장애물 탐지 범위를 확장하여 더 먼 거리에서 장애물을 감지하고 회피할 수 있도록 하여 안전성을 높임.



navi2.yaml tuning value 전략

속도관련 설정 Followpath

최대 선속도 (Max_Speed_XY)
기본값: 0.26m/s → 수정값: 0.13m/s



navi2.yaml tuning value 시행착오

시행착오

- cost_scaling_factor: 기본 값: 4.0 → 1차수정 값: 1.5 → 2차수정 값: 4.0
- Slam 파라미터 map_update_interval: 기본값: 0.5 → 기본값: 0.2
- 최대 선속도 (Max_Speed_XY) → 기본값: 0.26m/s → 수정값: 0.13m/s
- map_update_interval (기본값: 0.5) → (수정값: 0.2)

